Nginx Complete Learning Roadmap

Revision & Learning Path for Beginners

🖨 Nginx Complete Learning Roadmap

1. Core Concepts (Must Know)

These explain how Nginx works internally.

Concept	Purpose
Nginx	High-performance web server, reverse proxy, load balancer
Master Process	Reads config, starts and manages workers
Worker Process	Handles client requests
Event-Driven Architecture	Efficiently handles many connections
Non-Blocking I/O	Doesn’t wait for one request before serving another
Worker Connections	Maximum simultaneous connections per worker
Worker Processes	Number of worker processes (usually one per CPU core)
Keepalive	Reuses TCP connections to reduce overhead

2. Configuration Structure

These define how `nginx.conf` is organized.

```
nginx.conf
│
├── main
│
├── events
│
└── http
      │
      ├── upstream
      │
      └── server
            │
            └── location
```

Block	Purpose
main	Global settings
events	Worker and connection settings
http	HTTP configuration
upstream	Backend server group
server	Virtual host (website/app)
location	URL routing

3. Important Directives

These are configuration instructions.

Directive	Example	Purpose
listen	`listen 80;`	Port to listen on
server_name	`server_name example.com;`	Domain name
root	`/var/www/html`	Static file directory
alias	`/data/images`	Maps URL to another directory
index	`index index.html;`	Default page
proxy_pass	`http://localhost:3000`	Forward request to backend
return	`return 301 ...`	Redirect
rewrite	Rewrite URLs	URL rewriting
try_files	SPA routing	Check files before fallback

4. Routing Concepts

These determine where requests go.

Term	Purpose
location	Matches URL paths
Prefix Match	`/api`
Exact Match	`=`
Regex Match	`~`
Named Location	`@backend`

Example:

```
location /api {
    proxy_pass http://localhost:3000;
}

location /images {
    root /var/www/html;
}
```

5. Reverse Proxy Concepts

Term	Purpose
Reverse Proxy	Client talks only to Nginx
proxy_pass	Send request to backend
Proxy Headers	Pass client information
WebSocket Proxy	Support real-time connections

Example:

```
location /api {
    proxy_pass http://localhost:5000;
}
```

6. Load Balancing

Term	Purpose
Upstream	Group backend servers
Round Robin	Default balancing
Least Connections	Send to least busy server
IP Hash	Same client → same server
Health Checks	Detect failed servers

Example:

```
upstream backend {
    server localhost:3000;
    server localhost:3001;
}
```

7. Static File Serving

Feature	Purpose
root	Website directory
alias	Different filesystem path
MIME Types	Correct content type
index	Default file
Autoindex	Directory listing

8. Security Features

Feature	Purpose
SSL/TLS	HTTPS
HTTP → HTTPS Redirect	Force secure traffic
HSTS	Enforce HTTPS
Rate Limiting	Prevent abuse
Basic Authentication	Password protection
Hide Server Version	Improve security

9. Performance Features

Feature	Purpose
Gzip Compression	Smaller responses
Brotli Compression	Better compression (optional module)
Keepalive	Reuse connections
Caching	Faster responses
Buffering	Smooth data transfer
Sendfile	Faster file delivery

10. Logging & Monitoring

Feature	Purpose
Access Log	Record all requests
Error Log	Record errors
Log Format	Customize logs
Debug Logging	Troubleshooting

11. Common Use Cases

🌐 1. Static Website Hosting

```
Browser
   |
   ▼
 Nginx
   |
HTML/CSS/JS
```

Example: - Portfolio - Landing page - React production build

🚀 2. Reverse Proxy

```
Browser
   |
   ▼
 Nginx
   |
Node.js
```

Example: - Express API - NestJS - Django - Spring Boot

⚖️ 3. Load Balancer

```
         Nginx
       /   |   \
    App1  App2  App3
```

Distributes traffic across multiple servers.

🔒 4. SSL Termination

```
HTTPS
   |
 Nginx
   |
HTTP
   |
Backend
```

Nginx handles encryption so your backend can run on plain HTTP internally.

🎁 5. Serve React + Proxy API

```
Browser
```

```
     |
     ▼
  Nginx
  ├──────────────┐
  |              |
React Build   Node API
```

A very common MERN deployment pattern.

🌍 6. Multiple Websites (Virtual Hosts)

```
example.com
shop.com
blog.com
        |
     Nginx
```

One Nginx instance serves multiple domains.

📄 7. Static Asset Server

Serve: - Images - Videos - CSS - JavaScript - PDFs

without involving the backend.

🔁 8. URL Rewrite & Redirect

Examples: - HTTP → HTTPS - old-page → new-page - Trailing slash normalization

🚦 9. Rate Limiting

Protect APIs from excessive requests.

Example: - 100 requests/minute per IP

🌐 10. WebSocket Proxy

Supports real-time applications: - Chat apps - Notifications - Live dashboards - Multiplayer games

12. Important Configuration Files

File/Folder	Purpose
/etc/nginx/nginx.conf	Main configuration
/etc/nginx/conf.d/	Additional configs
/etc/nginx/sites-available/	Available virtual hosts (Debian/Ubuntu)
/etc/nginx/sites-enabled/	Enabled virtual hosts
/var/log/nginx/access.log	Access logs
/var/log/nginx/error.log	Error logs
/run/nginx.pid	Master process ID

🎯 Top 40 Nginx Interview Terms to Master

Core Architecture

1. Nginx
2. Master Process

3. Worker Process
4. Event-Driven Architecture
5. Non-Blocking I/O
6. Worker Processes
7. Worker Connections
8. Keepalive

Configuration

9. nginx.conf
10. Main Block
11. Events Block
12. HTTP Block
13. Server Block
14. Location Block
15. Upstream Block
16. Directive
17. Context
18. Include

Routing & Directives

19. listen
20. server_name
21. location
22. root
23. alias
24. index
25. proxy_pass
26. try_files
27. rewrite
28. return

Features

29. Reverse Proxy
30. Forward Proxy
31. Load Balancer
32. Virtual Host
33. Gzip Compression
34. Caching
35. SSL/TLS
36. Access Log
37. Error Log
38. Rate Limiting
39. WebSocket Proxy
40. HTTP → HTTPS Redirect